

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Уральский федеральный университет имени первого Президента России Б.Н. Ельцина»

Институт радиоэлектроники и информационных технологий

Лабораторная работа 3:

"Внедрение Dependency Injection и SQLAlchemy в Litestar"

Выполнила: студент магистратуры 1 курса Мансурова А.Р.

Екатеринбург, 2025

Цель работы: Освоить принципы Dependency Injection и интеграцию SQLAlchemy ORM веб-приложении на базе фреймворка Litestar написав CRUD.

Задачи:

- Изучить архитектурный паттерн Dependency Injection (DI) и его реализацию в фреймворке Litestar.
- Создать модель данных User с использованием SQLAlchemy ORM (асинхронного), включая поля: id, username, email, description, created_at, updated_at.
- Реализовать три слоя приложения:
- Repository (репозиторий): для выполнения операций с базой данных (CRUD).
- Service (сервис): для реализации бизнес-логики (например, проверка уникальности email, подготовка данных).
- Controller (контроллер): для обработки HTTP-запросов и валидации данных.
- Настроить асинхронное подключение к базе данных (например, SQLite) с использованием SQLAlchemy и aiosqlite.
- Реализовать CRUD-эндпоинты:
- GET /users — получить список пользователей (с пагинацией).
- GET /users/{id} — получить пользователя по ID.
- POST /users — создать нового пользователя.
- PUT /users/{id} — обновить пользователя.
- DELETE /users/{id} — удалить пользователя.
- Протестировать работу API с помощью HTTP-клиента (например, requests в Python или curl).
- Описать архитектуру приложения и взаимодействие между слоями.

Ход выполнения работы

Работу выполнять в том же репозитории, в котором выполнялась лабораторная работа 2. Для этого в репозитории необходимо добавить на последний коммит в мастере тэг lab_2. После завершения текущей лабораторной работы необходимо добавить тэг в мастер ветке lab_3. Для этого необходимо добавить тэг с помощью:

```
git tag <tag_name>
```

В лабораторной работе приведен темплейт простого CRUD, который необходимо реализовать

1. Репозиторий

Для работы с базой данных необходимо написать слой взаимодействия с базой данных для создания, обновления, получения и удаления пользователей.

```
from typing import Optional, List, Any
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import and_, select
from app.models.User import User
from app.User_schem import UserCreate, UserUpdate

class UserRepository:
    async def get_by_id(self, session: AsyncSession, user_id: str) -> Optional[User]:
        result = await session.execute(select(User).where(User.id == user_id))
        return result.scalar_one_or_none()

    async def get_by_filter(
        self,
        session: AsyncSession,
        count: int,
        page: int,
        **kwargs: Any
    ) -> List[User]:
        query = select(User)
```

```

filters = []
for key, value in kwargs.items():
    if not hasattr(User, key):
        continue
    column = getattr(User, key)
    if isinstance(value, str) and "%" in value:
        filters.append(column.like(value))
    else:
        filters.append(column == value)

if filters:
    query = query.where(and_(*filters))

offset = (page - 1) * count if page > 0 else 0
query = query.offset(offset).limit(count)

result = await session.execute(query)
return list(result.scalars().all())

async def create(self, session: AsyncSession, user_data: UserCreate) -> User:
    user_dict = user_data.model_dump(exclude_unset=True)
    db_user = User(**user_dict)
    session.add(db_user)
    await session.commit()
    await session.refresh(db_user)
    return db_user

async def update(
    self,
    session: AsyncSession,
    user_id: str,
    user_data: UserUpdate
) -> User:

```

```

db_user = await self.get_by_id(session, user_id)
if db_user is None:
    raise ValueError(f"User with id {user_id} not found")

update_data = user_data.model_dump(exclude_unset=True)
for field, value in update_data.items():
    if hasattr(db_user, field):
        setattr(db_user, field, value)

session.add(db_user)
await session.commit()
await session.refresh(db_user)
return db_user

async def delete(self, session: AsyncSession, user_id: str) -> None:
    db_user = await self.get_by_id(session, user_id)
    if db_user is not None:
        await session.delete(db_user)
        await session.commit()

```

2. Сервисный слой

Сервисный слой занимается бизнес логикой (в нём может быть проверки, интеграции с другими системами и тд), но на текущий момент сразу будет идти запись в базу данных.

```

from typing import Optional, List, Any
from sqlalchemy.ext.asyncio import AsyncSession

from app.models import User
from app.User_schem import UserCreate, UserUpdate
from app.repositories.user_repository import UserRepository

```

```

class UserService:
    def __init__(self, user_repository: UserRepository):
        self.user_repository = user_repository

    async def get_by_id(self, session: AsyncSession, user_id: str) -> Optional[User]:
        return await self.user_repository.get_by_id(session, user_id)

    async def get_by_filter(
        self,
        session: AsyncSession,
        count: int,
        page: int,
        **kwargs: Any
    ) -> List[User]:
        return await self.user_repository.get_by_filter(session, count=count, page=page, **kwargs)

    async def create(self, session: AsyncSession, user_data: UserCreate) -> User:
        return await self.user_repository.create(session, user_data)

    async def update(
        self,
        session: AsyncSession,
        user_id: str,
        user_data: UserUpdate
    ) -> User:
        return await self.user_repository.update(session, user_id, user_data)

    async def delete(self, session: AsyncSession, user_id: str) -> None:
        return await self.user_repository.delete(session, user_id)

```

3. Контроллер

Создадим контроллер для управления пользователем

```
from typing import List
```

```

from typing import List
from litestar import Controller, get, post, delete, put
from litestar.params import Parameter
from litestar.exceptions import NotFoundException
from sqlalchemy.ext.asyncio import AsyncSession

from app.User_schem import UserCreate, UserUpdate, UserResponse
from app.services.user_service import UserService


class UserController(Controller):
    path = "/users"

    @get()
    async def get_all_users(
        self,
        user_service: UserService,
        session: AsyncSession,
        count: int = Parameter(default=10, gt=0, le=100),
        page: int = Parameter(default=1, gt=0),
    ) -> List[UserResponse]:
        users = await user_service.get_by_filter(session, count=count, page=page)
        return [UserResponse.model_validate(user) for user in users]

    @get("/{user_id:str}")
    async def get_user_by_id(
        self,
        user_service: UserService,
        session: AsyncSession,
        user_id: str,
    ) -> UserResponse:
        user = await user_service.get_by_id(session, user_id)
        if not user:

```

```
        raise NotFoundException(detail=f"User with ID {user_id} not found")
    return UserResponse.model_validate(user)
```

```
@post()
```

```
async def create_user(
    self,
    user_service: UserService,
    session: AsyncSession,
    data: UserCreate, # ← ИМЯ "data" для тела запроса
) -> UserResponse:
    user = await user_service.create(session, data)
    return UserResponse.model_validate(user)
```

```
@delete("/{user_id:str}")
```

```
async def delete_user(
    self,
    user_service: UserService,
    session: AsyncSession,
    user_id: str,
) -> None:
    await user_service.delete(session, user_id)
```

```
@put("/{user_id:str}")
```

```
async def update_user(
    self,
    user_service: UserService,
    session: AsyncSession,
    user_id: str,
    data: UserUpdate, # ← ИМЯ "data"
) -> UserResponse:
    user = await user_service.update(session, user_id, data)
    return UserResponse.model_validate(user)
```

4. Главное приложение

Собираем приложение и добавляем зависимости в litestar

```
import os
import asyncio
from litestar import Litestar
from litestar.di import Provide
from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine
from sqlalchemy.orm import sessionmaker

from app.controllers.user_controller import UserController
from app.repositories.user_repository import UserRepository
from app.services.user_service import UserService

from app.models.User import User
from app.models.Address import Address
from app.models.Product import Product
from app.models.Order import Order
from app.models.Base import Base

# Настройка БД
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite+aiosqlite:///./test.db")
connect_args = {"check_same_thread": False} if DATABASE_URL.startswith("sqlite") else {}

engine = create_async_engine(DATABASE_URL, echo=True, connect_args=connect_args)
async_session_factory = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

# Провайдеры
async def provide_db_session() -> AsyncSession:
    async with async_session_factory() as session:
        yield session
```

```

def provide_user_repository() -> UserRepository:
    return UserRepository()

def provide_user_service(user_repository: UserRepository) -> UserService:
    return UserService(user_repository=user_repository)

app = Litestar(
    route_handlers=[UserController],
    dependencies={
        "session": Provide(provide_db_session),
        "user_repository": Provide(provide_user_repository, sync_to_thread=False),
        "user_service": Provide(provide_user_service, sync_to_thread=False),
    },
)

# Инициализация БД
async def create_tables():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

if __name__ == "__main__":
    asyncio.run(create_tables())
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

Запускаем приложение:


```

response = requests.get(f'{BASE_URL}/users?count=5&page=1")

print("\nСтатус-код : ", response.status_code)

if response.status_code == 200:
    users = response.json()
    print(f"Найдено пользователей (count=5): {len(users)}")
    for user in users:
        print(f"- ID: {user['id']}, Username: {user['username']}, Email: {user['email']}")
else:
    print("Ошибка при получении пользователей.", response.json())

if __name__ == "__main__":
    test_get_all_users()
    test_get_users_with_pagination()

```

результат теста

```

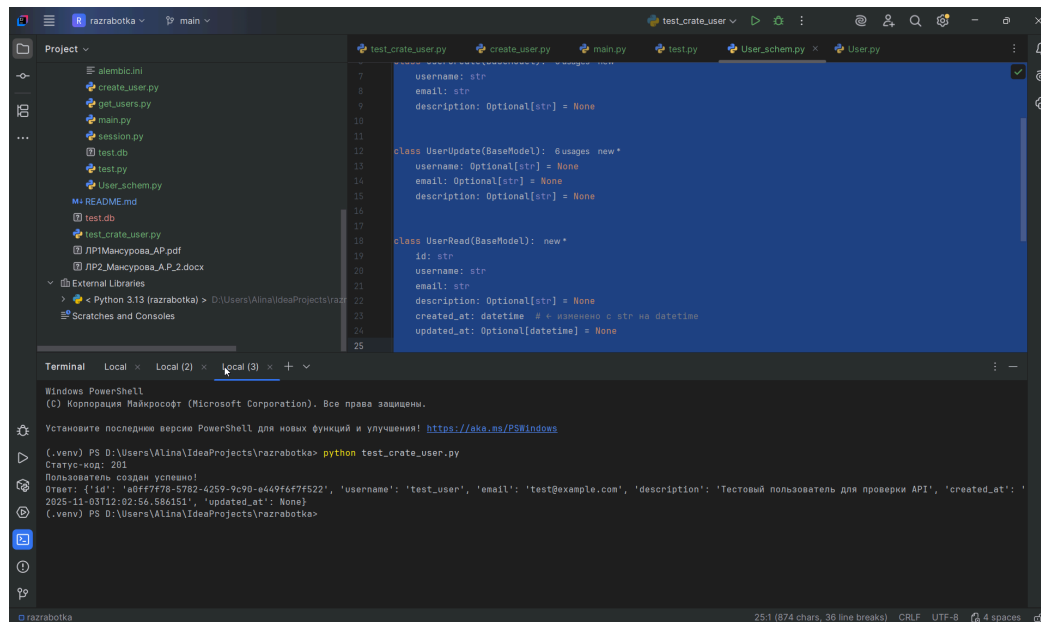
D:\Users\Aline\IdeaProjects\razrabotka\venv\Scripts\python.exe D:\Users\Aline\IdeaProjects\razrabotka\app\test.py
Статус-код: 200
Найдено пользователей: 10
- ID: f83b087b-ce58-4e1c-9984-da3e74ca3ecd, Username: Alice Johnson, Email: alice@example.com
- ID: 429a967b-b6a2-431b-a9df-51aa070210ec, Username: Bob Smith, Email: bob@example.com
- ID: bbc7f7b9-25d3-40b3-a04f-ffa6cfa1ec8e, Username: Charlie Brown, Email: charlie@example.com
- ID: 5230f5b0-ba7c-49a2-9a7d-dee2c1259075, Username: Diana Prince, Email: diana@example.com
- ID: 4b23bb54-c9c7-436b-8956-df73f49718e9, Username: Evan Davis, Email: evan@example.com
- ID: a8819eb2-296a-4527-b786-462fc211cf0a, Username: Aline Johnson, Email: alic@example.com
- ID: 5c1e2aed-f63f-462c-8b84-de80b1f93aea, Username: Greg Smith, Email: bb@example.com
- ID: e1d51d20-2b5e-4e40-bd7a-cda37a6a3394, Username: Idni Brown, Email: chalie@example.com
- ID: 5f1a8b3e-507e-41bf-8150-698ed2703ee8, Username: Dian Prince, Email: diaa@example.com
- ID: 35c53463-b99c-4bb8-b530-3e3a1b3eeb09, Username: Ivan Davis, Email: evn@example.com

Статус-код (с пагинацией): 200
Найдено пользователей (count=5): 5
- ID: f83b087b-ce58-4e1c-9984-da3e74ca3ecd, Username: Alice Johnson, Email: alice@example.com
- ID: 429a967b-b6a2-431b-a9df-51aa070210ec, Username: Bob Smith, Email: bob@example.com
- ID: bbc7f7b9-25d3-40b3-a04f-ffa6cfa1ec8e, Username: Charlie Brown, Email: charlie@example.com
- ID: 5230f5b0-ba7c-49a2-9a7d-dee2c1259075, Username: Diana Prince, Email: diana@example.com
- ID: 4b23bb54-c9c7-436b-8956-df73f49718e9, Username: Evan Davis, Email: evan@example.com

Process finished with exit code 0

```

Проверим создание пользователя



```
import requests
```

```
BASE_URL = "http://localhost:8000"
```

```
def test_create_user():
```

```
    user_data = {
```

```
        "username": "test_user",
```

```
        "email": "test@example.com",
```

```
        "description": "Тестовый пользователь для проверки API"
```

```
    }
```

```
    response = requests.post(
```

```
        f"{BASE_URL}/users",
```

```
        json=user_data
```

```
    )
```

```
    print("Статус-код:", response.status_code)
```

```
    if response.status_code == 201 or response.status_code == 200:
```

```
        user = response.json()
```

```
        print("Пользователь создан успешно!")
```

```
        print("Ответ:", user)
```

```
    return user
```

```
else:
    print("Ошибка при создании пользователя:", response.json())
    return None
if __name__ == "__main__":
    test_create_user()
```

*. Задание со звездочкой

Необходимо при запросе списка пользователей также возвращать количество пользователей в базе данных.

```
from typing import List
from litestar import Controller, get, post, delete, put
from litestar.params import Parameter
from litestar.exceptions import NotFoundException
from sqlalchemy.ext.asyncio import AsyncSession

from app.user_schem import UserCreate, UserUpdate, UserResponse, UsersListResponse
from app.services.user_service import UserService

class UserController(Controller):
    path = "/users"
```

@get()

```
async def get_all_users(
    self,
    user_service: UserService,
    session: AsyncSession,
    count: int = Parameter(default=10, gt=0, le=100),
    page: int = Parameter(default=1, gt=0),
) -> UsersListResponse:
    users = await user_service.get_by_filter(session, count=count, page=page)
    total_count = await user_service.get_total_count(session)
    return UsersListResponse(
        users=[UserResponse.model_validate(user) for user in users],
        total_count=total_count
    )
```

@get("/{user_id:str}")

```
async def get_user_by_id(
    self,
    user_service: UserService,
    session: AsyncSession,
    user_id: str,
) -> UserResponse:
    user = await user_service.get_by_id(session, user_id)
    if not user:
        raise NotFoundException(detail=f"User with ID {user_id} not found")
    return UserResponse.model_validate(user)
```

@post()

```
async def create_user(
    self,
    user_service: UserService,
    session: AsyncSession,
```

```

        data: UserCreate,
    ) -> UserResponse:
        user = await user_service.create(session, data)
        return UserResponse.model_validate(user)

    @delete("/{user_id:str}")
    async def delete_user(
        self,
        user_service: UserService,
        session: AsyncSession,
        user_id: str,
    ) -> None:
        user = await user_service.get_by_id(session, user_id)
        if not user:
            raise NotFoundException(detail=f"User with ID {user_id} not found")
        await user_service.delete(session, user_id)

    @put("/{user_id:str}")
    async def update_user(
        self,
        user_service: UserService,
        session: AsyncSession,
        user_id: str,
        data: UserUpdate,
    ) -> UserResponse:
        try:
            user = await user_service.update(session, user_id, data)
        except ValueError as e:
            raise NotFoundException(detail=str(e))
        return UserResponse.model_validate(user)

```

Запустим тест

D:\Users\Alina\IdeaProjects\razrabotka\.venv\Scripts\python.exe

D:\Users\Alina\IdeaProjects\razrabotka\test.py

Всего пользователей в системе: 16

ID: f83b087b-ce58-4e1c-9984-0a3e74ca3ecd

Username: Alice Johnson

Email: alice@example.com

Description: Frontend developer

Created: 2025-10-26T10:53:11.725921

ID: 429a967b-b6a2-431b-a9df-51aa070210ec

Username: Bob Smith

Email: bob@example.com

Description: Data scientist

Created: 2025-10-26T10:53:11.729153

ID: bbc7f7b9-25d3-40b3-a04f-ffa6cfe1ec8e

Username: Charlie Brown

Email: charlie@example.com

Description: DevOps engineer

Created: 2025-10-26T10:53:11.729656

ID: 5230f5b0-ba7c-49a2-9a7d-0ee2c1259075

Username: Diana Prince

Email: diana@example.com

Description: UX designer

Created: 2025-10-26T10:53:11.730345

ID: 4b23bb54-c9c7-436b-8956-df73f49718e9

Username: Evan Davis

Email: evan@example.com

Description: Backend developer

Created: 2025-10-26T10:53:11.730765

ID: a8819eb2-296a-4527-b786-462fc211cf0a

Username: Alina Johnson

Email: aice@example.com

Description: Frontend developer

Created: 2025-11-03T10:16:34.622964

ID: 5c1e2aed-f63f-462c-8b84-de80b1f93aea

Username: Greg Smith

Email: bb@example.com

Description: Data scientist

Created: 2025-11-03T10:16:34.627189

ID: e1d51d20-2b5e-4e40-bd7a-cda37a6a3394

Username: Idni Brown

Email: chalie@example.com

Description: DevOps engineer

Created: 2025-11-03T10:16:34.627522

ID: 5f1a8b3e-587e-41bf-8150-698ed27d3ee8

Username: Dian Prince

Email: diaa@example.com

Description: UX designer

Created: 2025-11-03T10:16:34.627980

ID: 35c53463-b99c-4bb8-b530-3e3a1b3eeb09

Username: Ivan Davis

Email: evn@example.com

Description: Backend developer

Created: 2025-11-03T10:16:34.628278

Process finished with exit code 0

Вопросы

1. Объясните принцип Dependency Injection (DI) своими словами. Какую проблему он решает в контексте разработки приложений и какие преимущества дает?

Dependency Injection (DI) — это паттерн проектирования, при котором объекты (зависимости) передаются в компонент (например, класс или функцию) извне, а не создаются внутри него.

Dependency Injection — это когда один объект не создаёт сам себе нужные ему "инструменты", а получает их извне, как будто ему их "внедряют".

Пример:

Вместо того чтобы контроллер сам создавал UserRepository, он получает его как параметр. Это как если тебе не нужно идти в магазин за инструментом, а тебе его просто дают.

Проблема, которую решает:

Если объект сам создаёт зависимости, он жёстко привязан к ним. Это сложно тестировать и изменять.

Преимущества:

Проще тестировать: можно подать "фальшивую" зависимость (мок).

Код гибче: легко заменить одну реализацию на другую (например, разные базы данных).

Чище и понятнее: каждый объект занимается своим делом, а зависимости получает сверху.

2. Каковы основные обязанности каждого из трёх слоев приложения (Repository, Service, Controller)? Почему такое разделение считается хорошей практикой? Что произойдет, если объединить логику репозитория и контроллера?

Обязанности:

Controller (контроллер): обрабатывает HTTP-запросы и отвечает за валидацию данных.

Service (сервис): содержит бизнес-логику, например, проверки, транзакции, отправку писем.

Repository (репозиторий): работает с базой данных — делает SELECT, INSERT, UPDATE, DELETE.

Почему это хорошо:

Каждый слой делает свою работу, и не мешает другим.

Легче тестировать, читать и изменять код.

Можно использовать один и тот же сервис в разных контроллерах.

Что будет, если объединить репозиторий и контроллер:

Контроллер станет "толстым", в нём будет и HTTP-логика, и работа с БД. Сложнее тестировать. Код труднее поддерживать.

3. Объясните жизненный цикл зависимости в Litestar. Как именно создается и когда уничтожается экземпляр сессии базы данных при обработке одного HTTP-запроса?

Жизненный цикл сессии:

При начале обработки запроса Litestar создаёт сессию (например, через `provide_db_session`).

Эта сессия передаётся во все нужные слои (контроллер → сервис → репозиторий).

Все изменения в БД накапливаются в этой сессии.

Если всё прошло успешно, сессия делает `commit()` (сохраняет изменения).

Если была ошибка, делается `rollback()` (откат).

После завершения запроса сессия закрывается и уничтожается.

Зачем это нужно:

Каждый запрос = одна транзакция, и всё связано между собой.

Сессия не "живёт" дольше, чем нужно — безопасно и эффективно.

4. Что такое `async/await` и зачем они используются в данном приложении? Как асинхронность влияет на производительность при работе с базой данных?

`async` — значит, функция может ждать выполнения каких-то действий (например, запроса к БД), не блокируя весь поток.

`await` — ожидание результата от асинхронной операции.

Зачем это нужно:

Если сервер делает запрос к БД, он не ждёт вхолостую, а может заняться другими запросами.

Это позволяет обрабатывать больше запросов одновременно, даже если они связаны с ожиданием.

Как влияет на производительность:

Быстрее отрабатывают запросы, связанные с I/O (например, база данных, сетевые вызовы).

Экономия ресурсов: не создаются лишние треды.

5. Почему в сигнатурах методов UserRepository первым аргументом передается session? Почему бы не создать его внутри репозитория? Кто и когда должен вызывать session.commit() или session.rollback()?

Почему передаём session извне:

Чтобы все действия в одном запросе были в одной транзакции.

Если репозиторий сам создаст сессию — это будет другая транзакция, и изменения не будут связаны с остальными.

Кто вызывает commit() / rollback():

Litestar автоматически делает commit() при успешном завершении запроса.

Если была ошибка — делает rollback().

6. Для чего в методе get_by_filter используется пагинация (count и page)?

Зачем нужна пагинация:

Избежать перегрузки сервера и клиента: если в БД 100 000 пользователей, и вы запросите их всех — это медленно и непрактично.

Удобство клиента: пользователь может загружать данные частями (например, по 10 на странице).

Экономия памяти: сервер возвращает только нужное количество записей.

Как работает:

count — сколько записей вернуть.

page — какую "страницу" вернуть.

offset = (page - 1) * count — сдвиг от начала.

7. В текущей реализации UserService практически не содержит бизнес-логики и является "прокси" для UserRepository. Приведите пример конкретной бизнес-логики (например, проверка уникальности email, хеширование пароля, отправка приветственного письма), которую можно было бы добавить в сервисный слой.

Примеры бизнес-логики:

Проверить, что email уникален (нет ли уже пользователя с таким email).

Хешировать пароль перед сохранением.

Отправить приветственное письмо после создания пользователя.

Проверить, не заблокирован ли пользователь.

Инкрементировать счётчик созданных пользователей.

8. Какие HTTP-статусы должны возвращать каждый из эндпоинтов (get, post, put, delete) в различных сценариях (успех, ошибка, не найден)? Обоснуйте свой выбор.

GET /users/{id}	Пользователь найден	200 OK	Успешный запрос, объект возвращён.
GET /users/{id}	Пользователь не найден	404 Not Found	Объекта не существует.
POST /users	Пользователь создан	201 Created	Ресурс успешно создан.
POST /users	Ошибка валидации	Unprocessable Entity	Данные не прошли проверку.
PUT /users/{id}	Пользователь обновлён	200 OK	Успешно обновлено.
PUT /users/{id}	Не найден	404 Not Found	Нечего обновлять.
DELETE /users/{id}	Успешно удалён	204 No Content	Успешно, но тело не возвращается.
DELETE /users/{id}	Не найден	404 Not Found	Нечего удалять.

Обоснование:

- 201 — используется при создании.
- 404 — если объект не найден.

- 422 — если ошибка валидации данных.
- 204 — если удаление прошло успешно, но возвращать нечего.